

AWS Assignment

AWS Database Services

RDS, Aurora, DynamoDB & Redshift

XYZ Corporation – step-by-step implementation with screenshots

Assignment	Service	Main resource
1	Amazon RDS – MariaDB	xyz-mariadb-db (db.t3.micro), clients: MySQL Workbench and Linux EC2
2	Amazon Aurora (MySQL)	xyz-aurora-cluster with 1 writer and 2 readers in different AZs
3	Amazon DynamoDB	xyz-dynamodb-table (partition key ID), 5 items, backup, delete
4	Amazon Redshift	xyz-redshift-cluster (ra3.large), data loaded with Query Editor v2

Region: US East (N. Virginia) – us-east-1 for all assignments.

Contents

Assignment 1: Amazon RDS – MariaDB.....	3
Step 1: Create the MariaDB RDS database.....	3
Step 2: Copy the database endpoint.....	3
Step 3: Allow database traffic in the RDS security group.....	4
Step 4: Launch the Linux EC2 client.....	5
Step 5: Connect using a SQL client for Windows (MySQL Workbench).....	5
Step 6: Connect from the Linux EC2 instance.....	6
Assignment 2: Amazon Aurora with Read Replicas.....	8
Step 1: Create the Aurora DB cluster.....	8
Step 2: Add the second read replica.....	8
Step 3: Verify the two read replicas.....	9
Assignment 3: Amazon DynamoDB.....	11
Step 1: Create the DynamoDB table.....	11
Step 2: Add five items.....	12
Step 3: Take an on-demand backup.....	13
Step 4: Delete the table.....	14
Step 5: Confirm the backup is still available.....	15
Assignment 4: Amazon Redshift.....	17
Step 1: Create the Redshift cluster.....	17
Step 2: Open Query Editor v2 and connect.....	17
Step 3: Load data.....	18
Step 4: Query the data.....	18
Conclusion.....	19

Assignment 1: Amazon RDS – MariaDB

Problem statement: XYZ Corporation's application requires a SQL service that can store data which can be retrieved when required. Implement a suitable RDS engine.

Tasks:

1. Create a MariaDB engine based RDS database.
2. Connect to the DB using: (a) a SQL client for Windows, (b) a Linux based EC2 instance.

Step 1: Create the MariaDB RDS database

1. Open **Aurora and RDS → Databases → Create database** and choose **Standard create**.
2. Engine type: **MariaDB**. Template: **Free tier / Sandbox**.
3. DB instance identifier: **xyz-mariadb-db**. Master username: **admin**. Set a master password.
4. Instance class: **db.t3.micro**. Storage: General Purpose SSD (20 GiB).
5. Connectivity: default VPC, **Public access: Yes** (needed for the Windows client), new VPC security group **XYZ-RDS-SG**, Availability Zone **us-east-1a**.
6. Click **Create database** and wait until the status is **Available**.

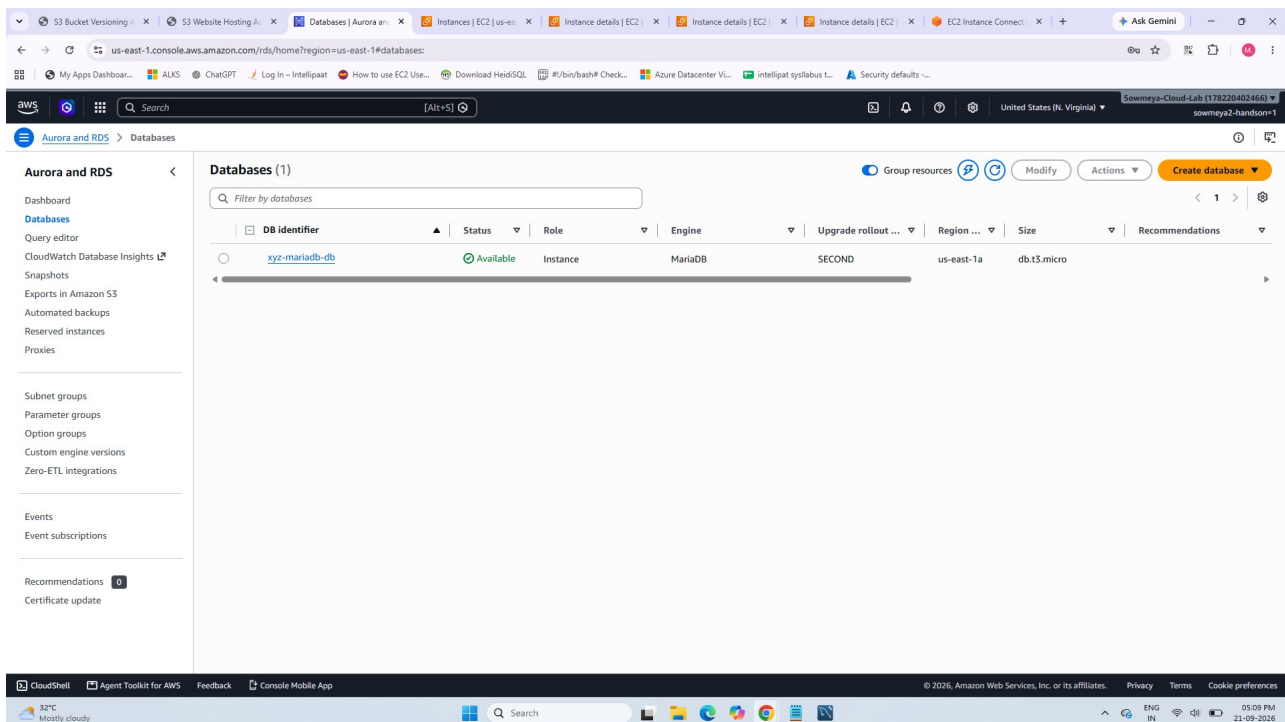


Figure 1: xyz-mariadb-db is Available (MariaDB, db.t3.micro, us-east-1a)

Step 2: Copy the database endpoint

Open **xyz-mariadb-db → Connectivity & security**. The endpoint and the ready-made connection commands are shown under **Code snippets**:

```
Endpoint : xyz-mariadb-db.ckb6yq8asxc4.us-east-1.rds.amazonaws.com
Port      : 3306
User      : admin
```

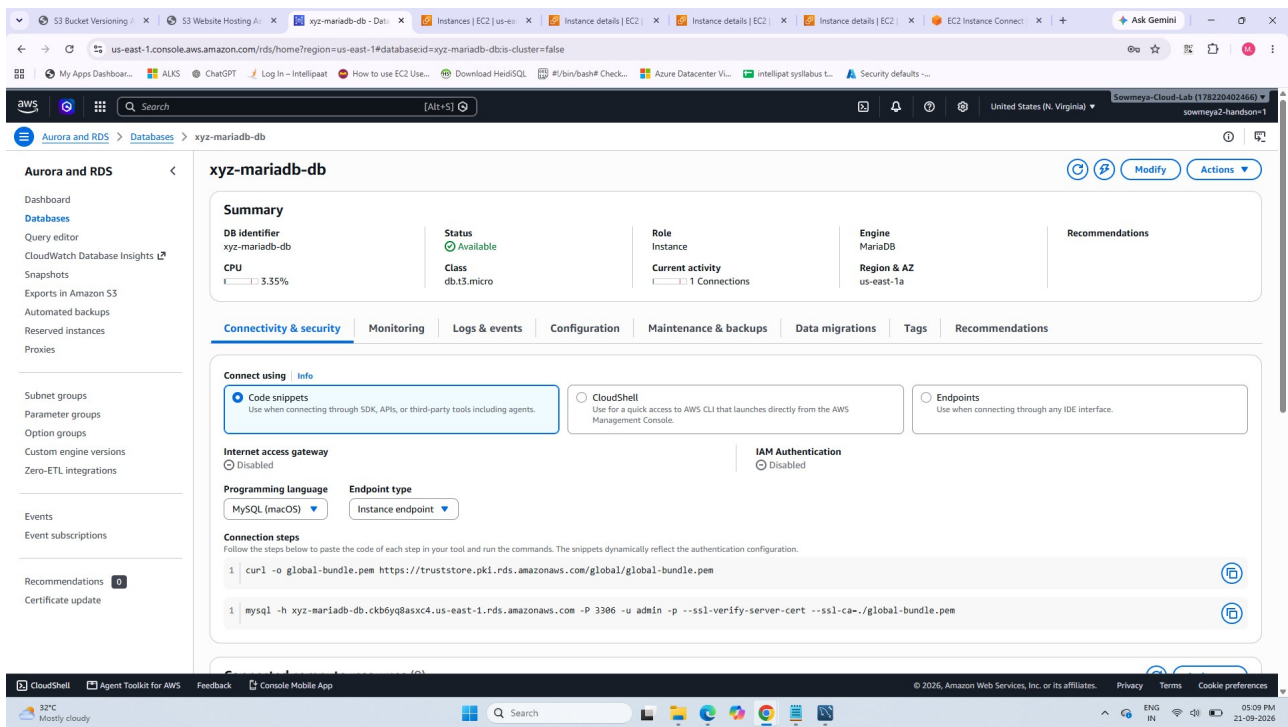


Figure 2: Connectivity & security – endpoint and connection snippet for xyz-mariadb-db

Step 3: Allow database traffic in the RDS security group

1. Open security group XYZ-RDS-SG → Edit inbound rules.
2. Add MySQL/Aurora (TCP 3306) from the EC2 client's security group sg-0e16dd40b14ff6be8 (launch-wizard-1).
3. Add MySQL/Aurora (TCP 3306) from your PC's IP so MySQL Workbench can connect (the screenshot shows 0.0.0.0/0; choosing My IP is safer).

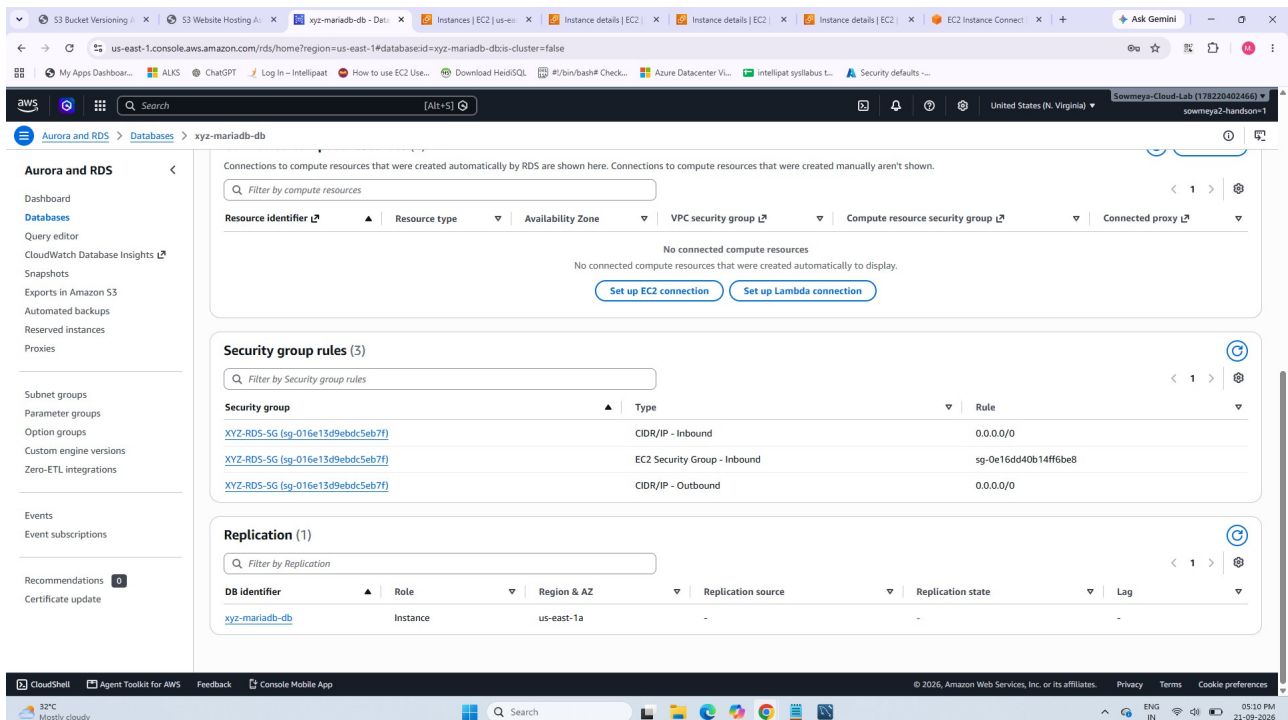


Figure 3: Security group rules on XYZ-RDS-SG – inbound from 0.0.0.0/0 and from the EC2 security group

Step 4: Launch the Linux EC2 client

1. **EC2 → Launch instances.** Name: **XYZ-Linux-DB-Client**. AMI: **Amazon Linux 2023**. Type: **t3.micro**.
2. Use the same VPC as the database, enable a public IP, and allow **SSH (22)** in its security group (launch-wizard-1).

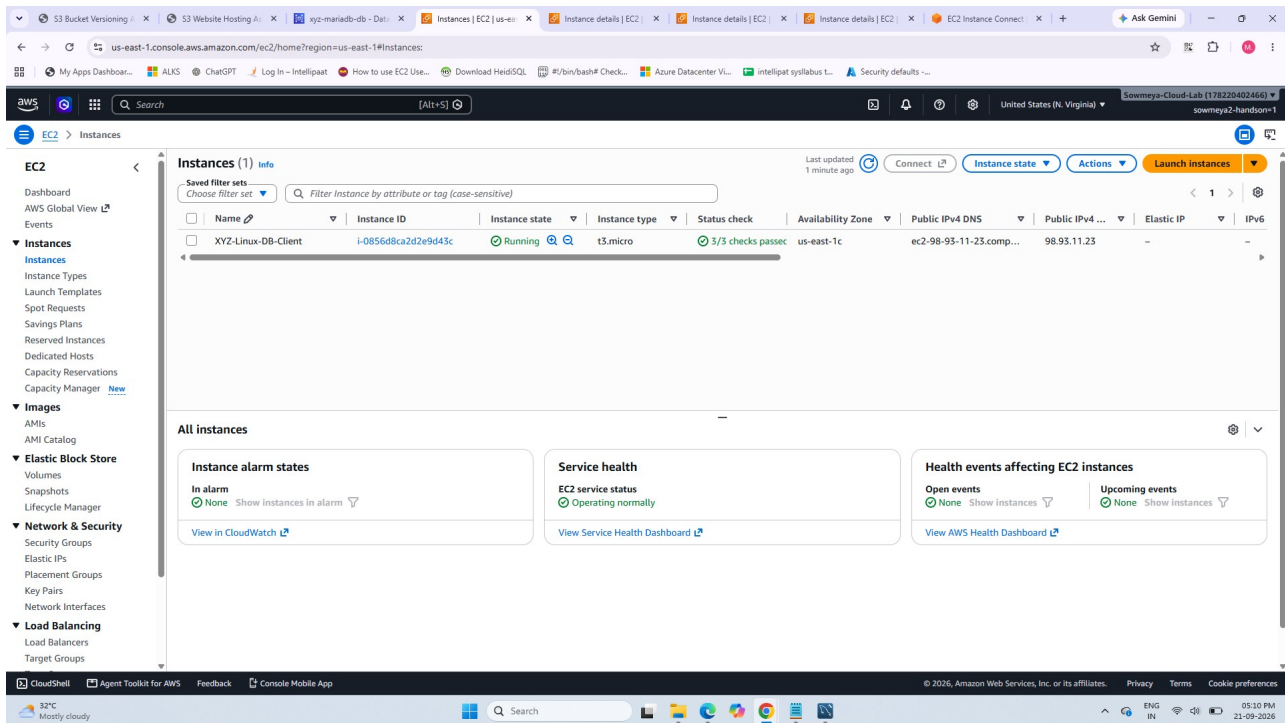


Figure 4: XYZ-Linux-DB-Client running (t3.micro, us-east-1c, public IP 98.93.11.23)

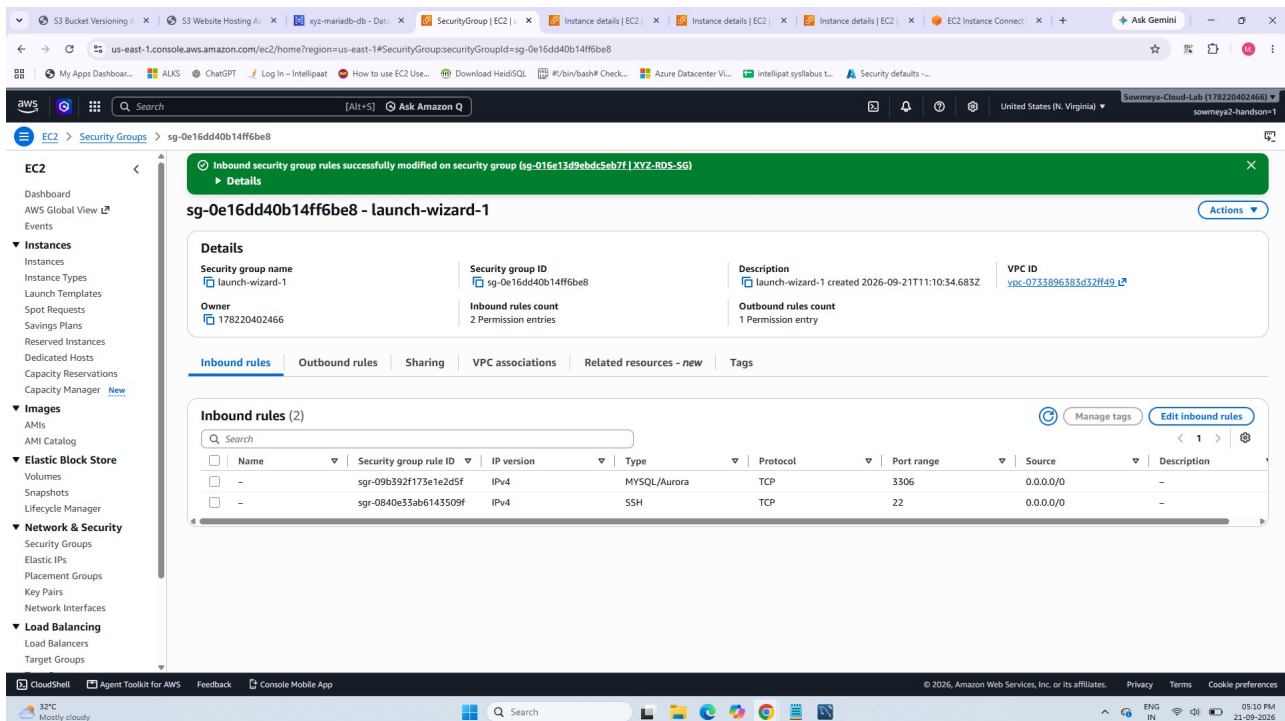


Figure 5: EC2 security group launch-wizard-1 – inbound SSH 22 and MySQL 3306

Step 5: Connect using a SQL client for Windows (MySQL Workbench)

1. Download the RDS CA certificate **global-bundle.pem** from <https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem>.

2. Open **MySQL Workbench** and click + next to **MySQL Connections**.
3. Connection Name: **XYZ-MariaDB-RDS**. Hostname: the RDS endpoint. Port: **3306**. Username: **admin**.
4. SSL tab: Use **SSL Require** and set SSL CA File to **global-bundle.pem** (this RDS instance only accepts secure connections).
5. Click **Test Connection**, enter the password, then **OK**. Open the connection and run **SHOW DATABASES;**.

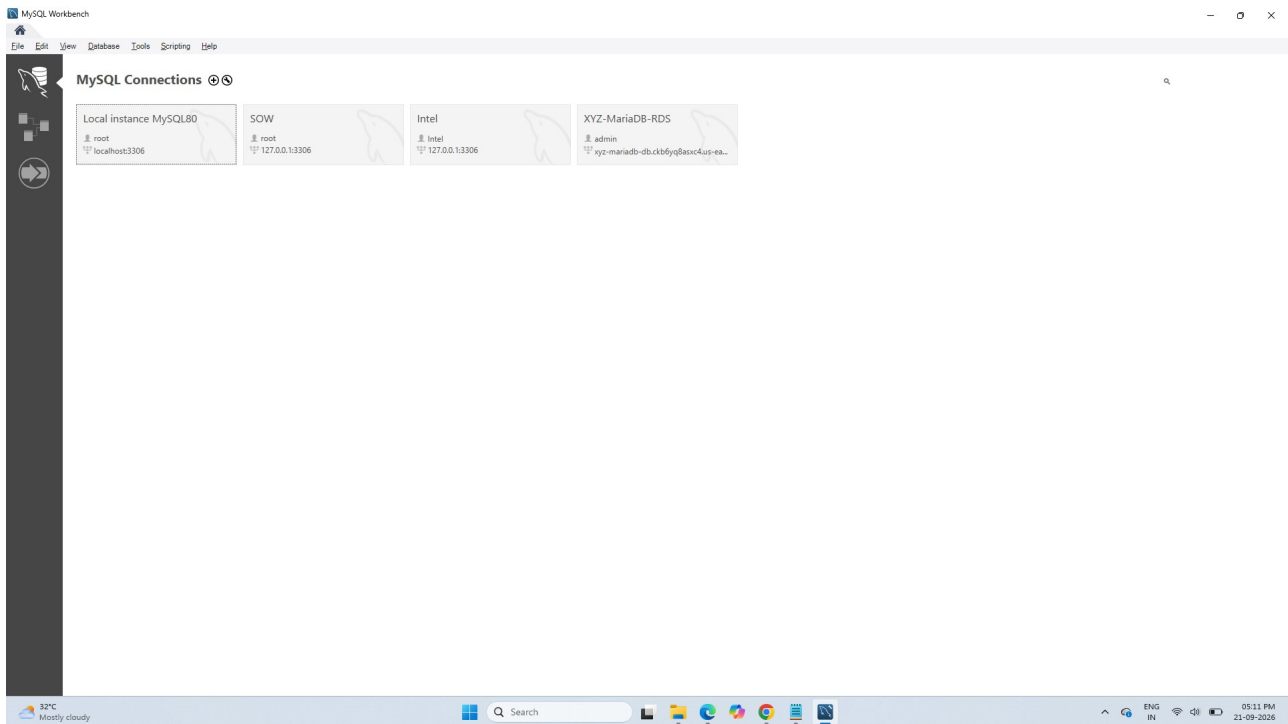


Figure 6: MySQL Workbench with the XYZ-MariaDB-RDS connection (user admin, RDS endpoint)

Step 6: Connect from the Linux EC2 instance

1. Select **XYZ-Linux-DB-Client** → **Connect** → **EC2 Instance Connect** → **Connect**.
2. Install the MariaDB client, download the CA bundle and connect with SSL:

```
sudo dnf install -y mariadb105
mariadb --version
curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
mariadb -h xyz-mariadb-db.ckb6yq8asxc4.us-east-1.rds.amazonaws.com -P 3306 \
-u admin -p --ssl-ca=./global-bundle.pem --ssl-verify-server-cert

SHOW DATABASES;
SELECT USER(), DATABASE(), VERSION();
```

```

Last login: Mon Sep 21 11:19:12 2026 from 18.206.107.28
[ec2-user@ip-172-31-16-208 ~]$ mariadb --version
mariadb Ver 15.1 Distrib 10.5.25-MariaDB, for Linux (x86_64) using Editline wrapper
[ec2-user@ip-172-31-16-208 ~]$ mariadb -h xyz-mariadb-db-ckb6yq8asxc4.us-east-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
ERROR 2002 (HY000): Can't connect to MySQL server on 'xyz-mariadb-db-ckb6yq8asxc4.us-east-1.rds.amazonaws.com' (115)
[ec2-user@ip-172-31-16-208 ~]$ mariadb -h xyz-mariadb-db-ckb6yq8asxc4.us-east-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
ERROR 3159 (08004): Connections using insecure transport are prohibited while --require_secure_transport=ON
[ec2-user@ip-172-31-16-208 ~]$ curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 161.5k 100 161.5k 0 0 5.72M 0
[ec2-user@ip-172-31-16-208 ~]$ mariadb -h xyz-mariadb-db-ckb6yq8asxc4.us-east-1.rds.amazonaws.com -P 3306 -u admin -p --ssl-ca=/global-bundle.pem --ssl-verify-server-cert
Enter password:
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MariaDB connection id is 132
Server version: 11.8.8-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| innodb |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.008 sec)

MariaDB [(none)]> SELECT USER(), DATABASE(), VERSION();
+-----+-----+-----+
| USER() | DATABASE() | VERSION() |
+-----+-----+-----+
| admin@172.31.16.208 | NULL | 11.8.8-MariaDB-log |
+-----+-----+-----+
1 row in set (0.001 sec)

MariaDB [(none)]>

```

i-0856d8ca2d2e9d43c (XYZ-Linux-DB-Client)
PublicIPs: 98.93.11.23 PrivateIPs: 172.31.16.208

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

32°C Mostly cloudy ENG IN 05:11 PM 21-09-2026

Figure 7: Connected from EC2 to MariaDB 11.8.8 over SSL – SHOW DATABASES and SELECT USER() output

Troubleshooting seen in the screenshot: ERROR 2002 (115) means the connection timed out – check the security group rule for port 3306. **ERROR 3159** means the server requires SSL (`require_secure_transport=ON`) – connect with `--ssl-ca` as shown above.

Assignment 2: Amazon Aurora with Read Replicas

Problem statement: XYZ Corporation's application requires a SQL service that can store data which can be retrieved when required. Implement a suitable RDS engine.

Tasks:

1. Create an Aurora DB engine based RDS database.
2. Create 2 read replicas in different Availability Zones for better availability.

Step 1: Create the Aurora DB cluster

1. **Aurora and RDS → Create database → Standard create.** Engine: **Aurora (MySQL Compatible).**
2. DB cluster identifier: **xyz-aurora-cluster**. Set the master username and password.
3. Instance configuration: **Burstable classes – db.t4g.medium.**
4. Availability & durability: **Create an Aurora Replica or Reader node in a different AZ.** This creates the first reader.
5. Choose the VPC and security group, then click **Create database.**

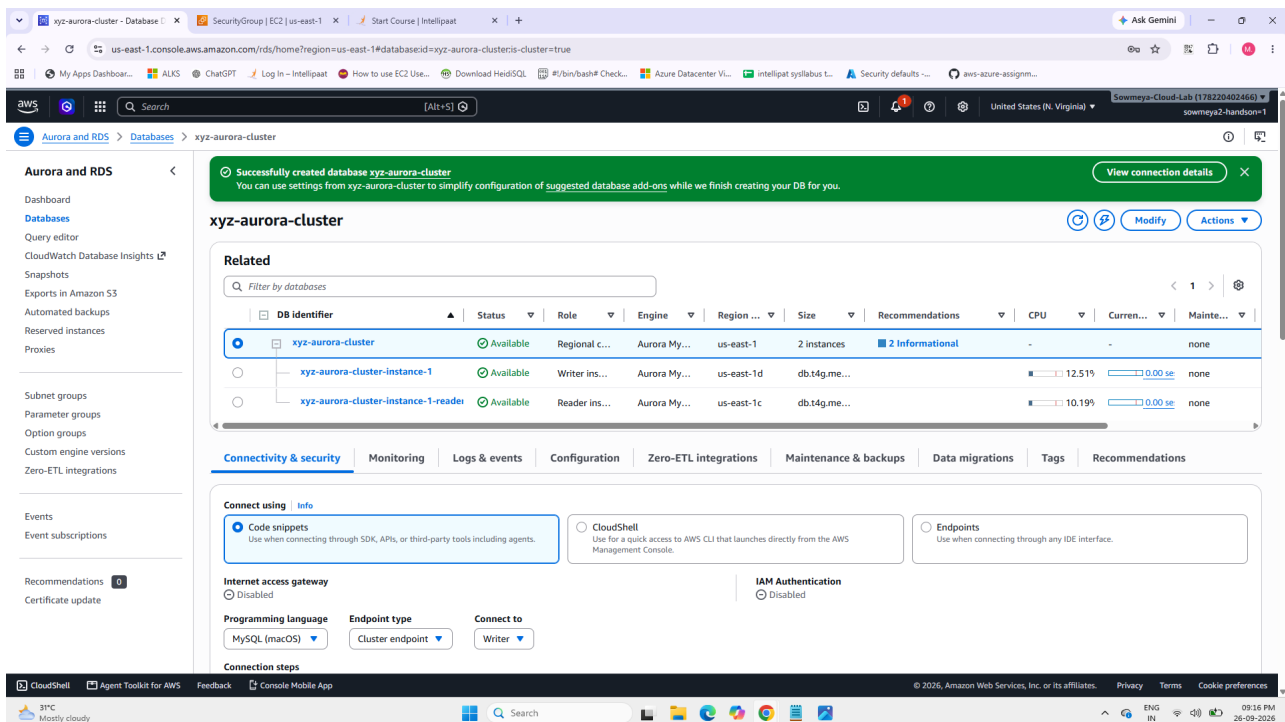


Figure 8: xyz-aurora-cluster created – writer in us-east-1d and first reader in us-east-1c

Step 2: Add the second read replica

1. Select **xyz-aurora-cluster** and choose **Actions → Add reader**.
2. DB instance identifier: **xyz-aurora-cluster-instance-2**. Instance class: **db.t4g.medium**.
3. Availability Zone: **us-east-1b** (different from the writer and the first reader). Click **Add reader**.

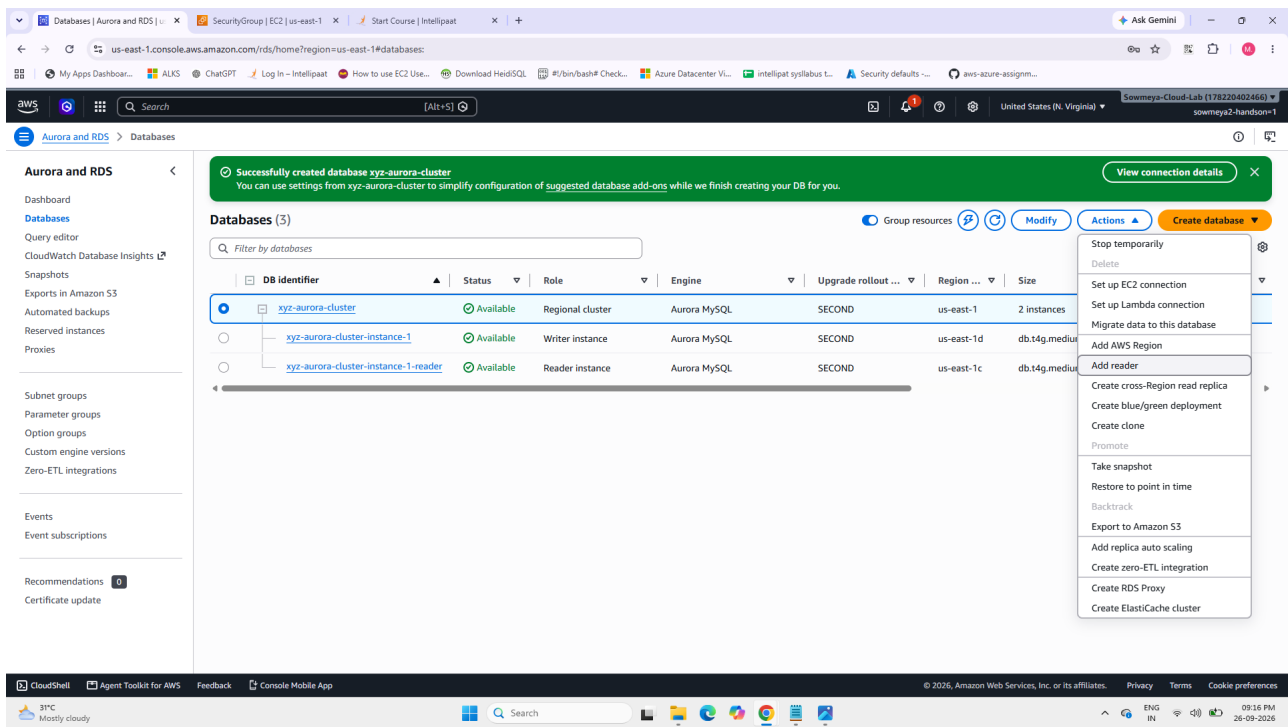


Figure 9: Actions → Add reader on xyz-aurora-cluster

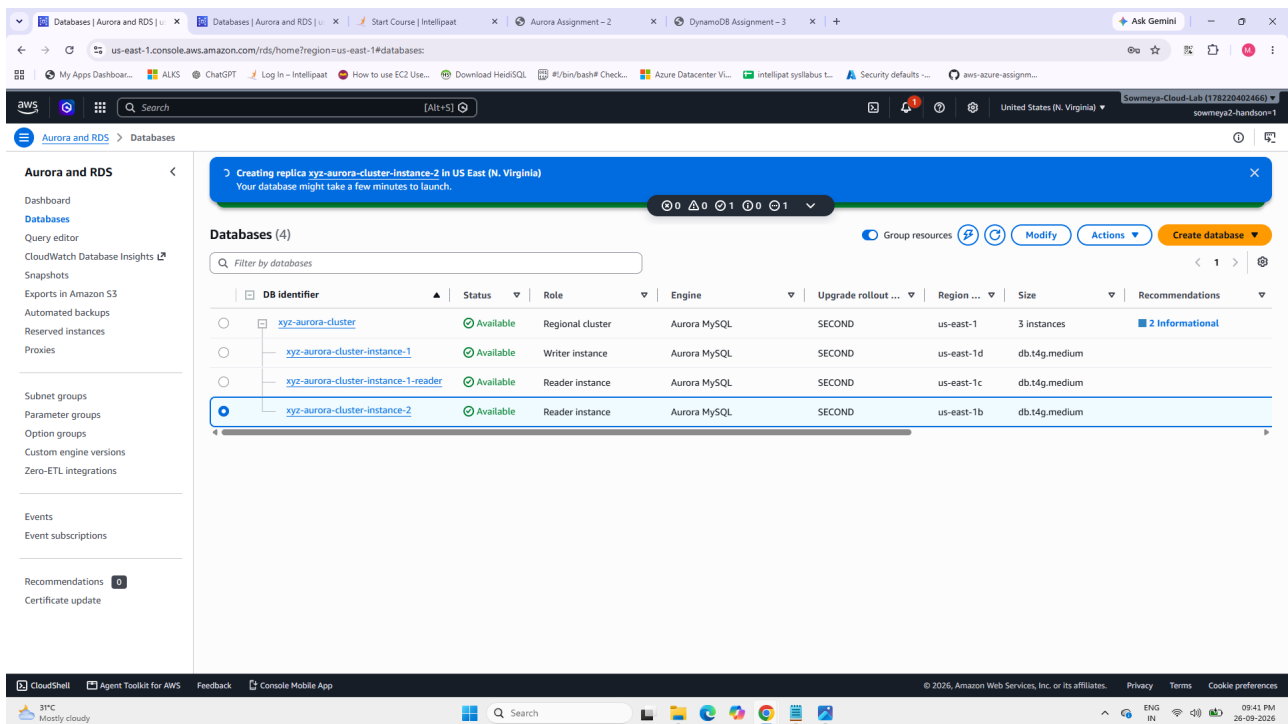


Figure 10: Creating replica xyz-aurora-cluster-instance-2 in us-east-1b

Step 3: Verify the two read replicas

The cluster now has 3 instances, each in a different Availability Zone:

DB instance	Role	Availability Zone	Class
xyz-aurora-cluster-instance-1	Writer	us-east-1d	db.t4g.medium
xyz-aurora-cluster-instance-1-reader	Reader (replica 1)	us-east-1c	db.t4g.medium
xyz-aurora-cluster-instance-2	Reader (replica 2)	us-east-1b	db.t4g.medium

The screenshot displays the AWS Management Console for the 'xyz-aurora-cluster'. A green notification banner at the top states: 'Successfully created database xyz-aurora-cluster. You can use settings from xyz-aurora-cluster to simplify configuration of suggested database add-ons while we finish creating your DB for you.' Below this, the 'Databases (4)' section shows a table of instances. The table has columns for DB identifier, Status, Role, Engine, Upgrade rollout, Region, Size, and Recommendations. The instances are:

DB identifier	Status	Role	Engine	Upgrade rollout	Region	Size	Recommendations
xyz-aurora-cluster	Available	Regional cluster	Aurora MySQL	SECOND	us-east-1	3 instances	2 Informational
xyz-aurora-cluster-instance-1	Available	Writer instance	Aurora MySQL	SECOND	us-east-1d	db.t4g.medium	
xyz-aurora-cluster-instance-1-reader	Available	Reader instance	Aurora MySQL	SECOND	us-east-1c	db.t4g.medium	
xyz-aurora-cluster-instance-2	Available	Reader instance	Aurora MySQL	SECOND	us-east-1b	db.t4g.medium	

The left sidebar shows the 'Aurora and RDS' navigation menu with options like Dashboard, Databases, Query editor, CloudWatch Database Insights, Snapshots, Exports in Amazon S3, Automated backups, Reserved instances, Proxies, Subnet groups, Parameter groups, Option groups, Custom engine versions, Zero-ETL integrations, Events, Event subscriptions, Recommendations (0), and Certificate update. The bottom status bar shows the AWS logo, CloudShell, Agent Toolkit for AWS, Feedback, Console Mobile App, and copyright information for Amazon Web Services, Inc. or its affiliates.

Figure 11: xyz-aurora-cluster with 3 instances – 1 writer and 2 readers in us-east-1d, 1c and 1b

Why this helps: If the writer or its AZ fails, Aurora promotes one of the readers to writer, and read traffic can be spread across both readers using the cluster reader endpoint.

Assignment 3: Amazon DynamoDB

Problem statement: XYZ Corporation's application requires a database service that can store data which can be retrieved when required. Implement a suitable service.

Tasks:

1. Create a DynamoDB table with partition key as ID.
2. Add 5 items to the DynamoDB table.
3. Take a backup and delete the table.

Step 1: Create the DynamoDB table

1. Open **DynamoDB → Tables → Create table**.
2. Table name: **xyz-dynamodb-table**. Partition key: **ID** (type **String**). No sort key.
3. Table settings: **Default settings** (On-demand capacity). Click **Create table**.

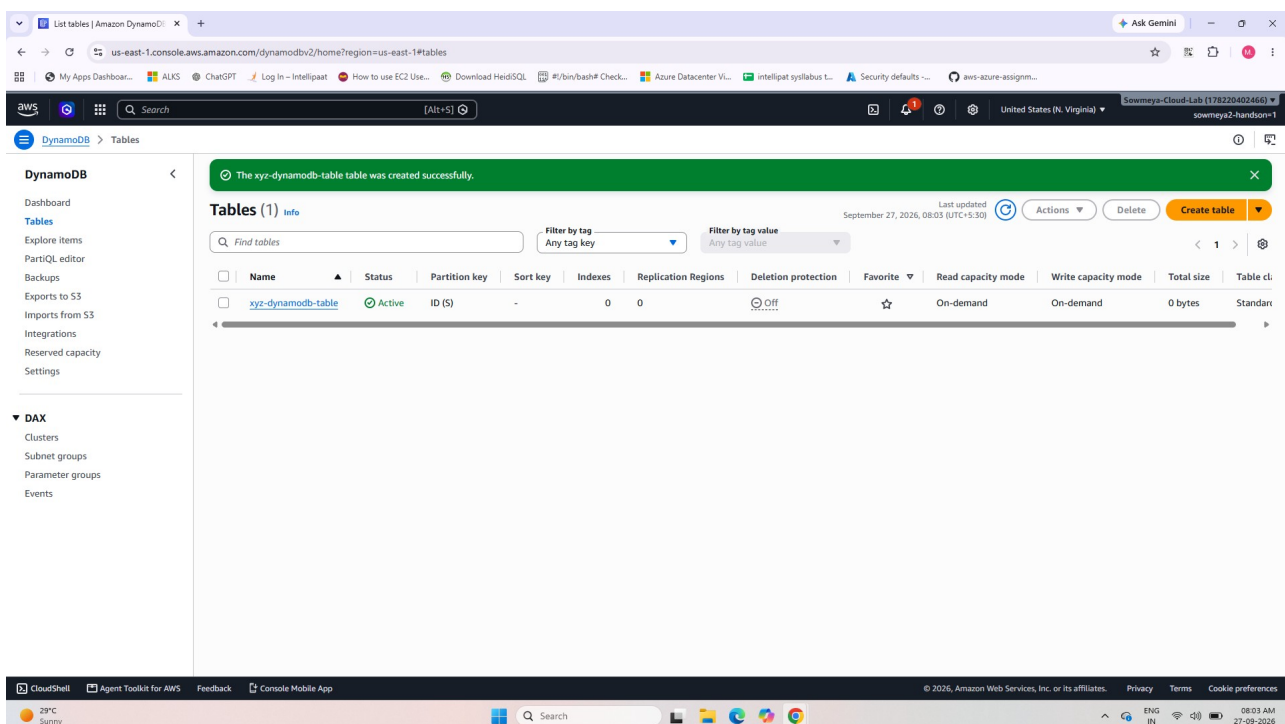


Figure 12: xyz-dynamodb-table created – partition key ID (String), On-demand, Active

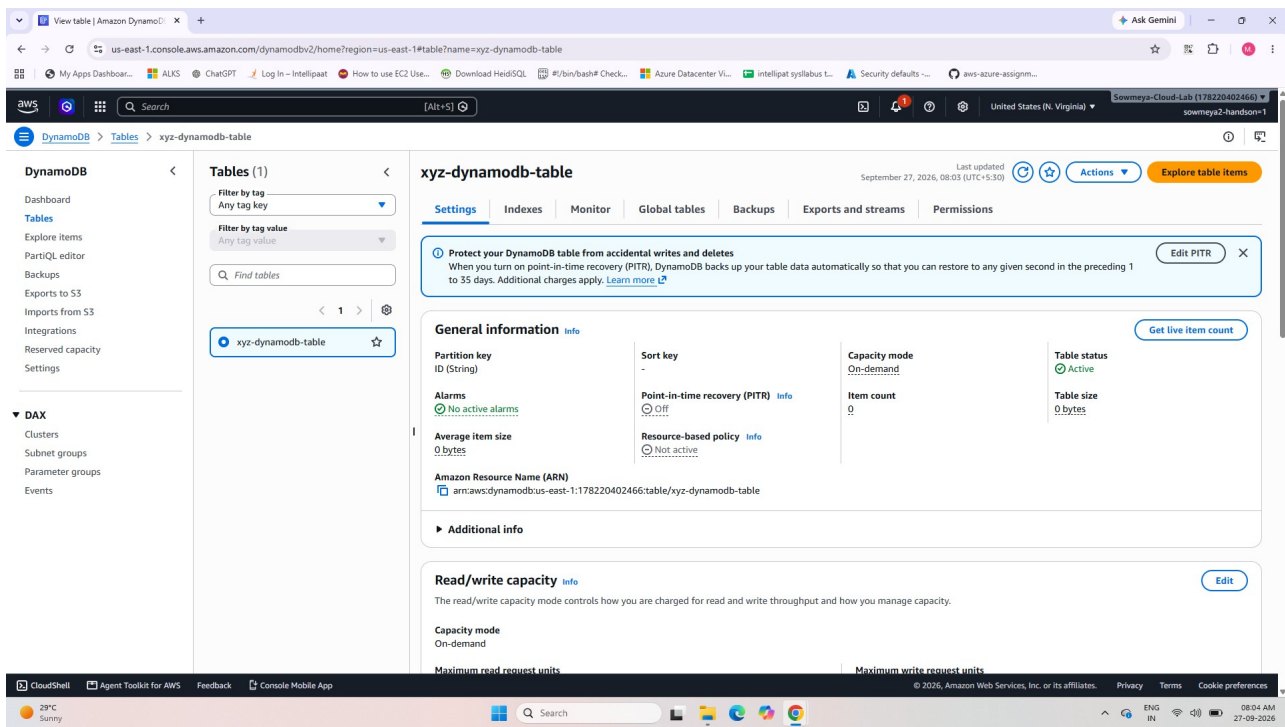


Figure 13: Table details – partition key ID (String), capacity mode On-demand

Step 2: Add five items

1. Click **Explore table items** → **Create item**.
2. Enter **ID** and add attributes **Name** and **Department** (String), then **Create item**. Repeat for all five items.

ID	Name	Department
1	Ravi	IT
2	Priya	HR
3	Arun	Finance
4	Meena	Sales
5	Siva	Devops

Example item in JSON view:

```
{
  "ID": { "S": "1" },
  "Name": { "S": "Ravi" },
  "Department": { "S": "IT" }
}
```

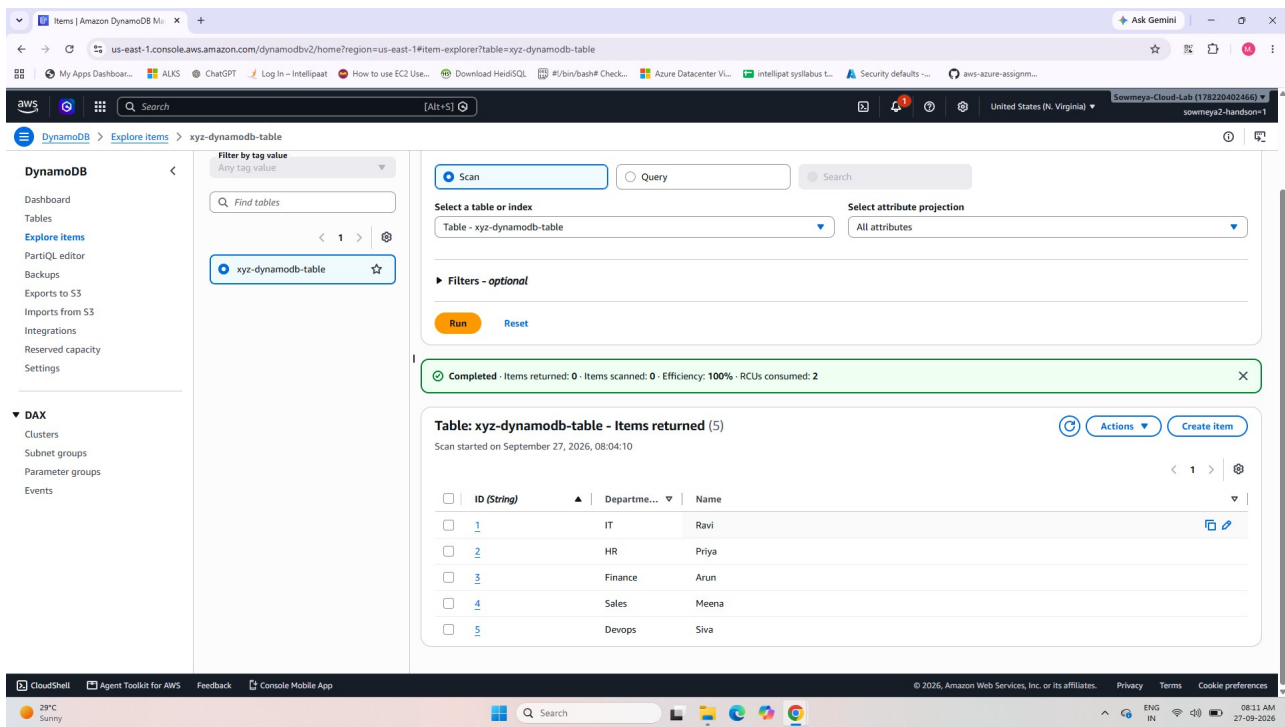


Figure 14: Scan of xyz-dynamodb-table – 5 items returned

Step 3: Take an on-demand backup

1. Open the table and choose **Actions** → **Create backup**.
2. Give the backup a name and click **Create backup**. Because **Advanced features with AWS Backup** is activated, the backup is handled by AWS Backup.
3. Click **View details** to open **AWS Backup** → **Jobs** and wait until the job status is **Completed**.

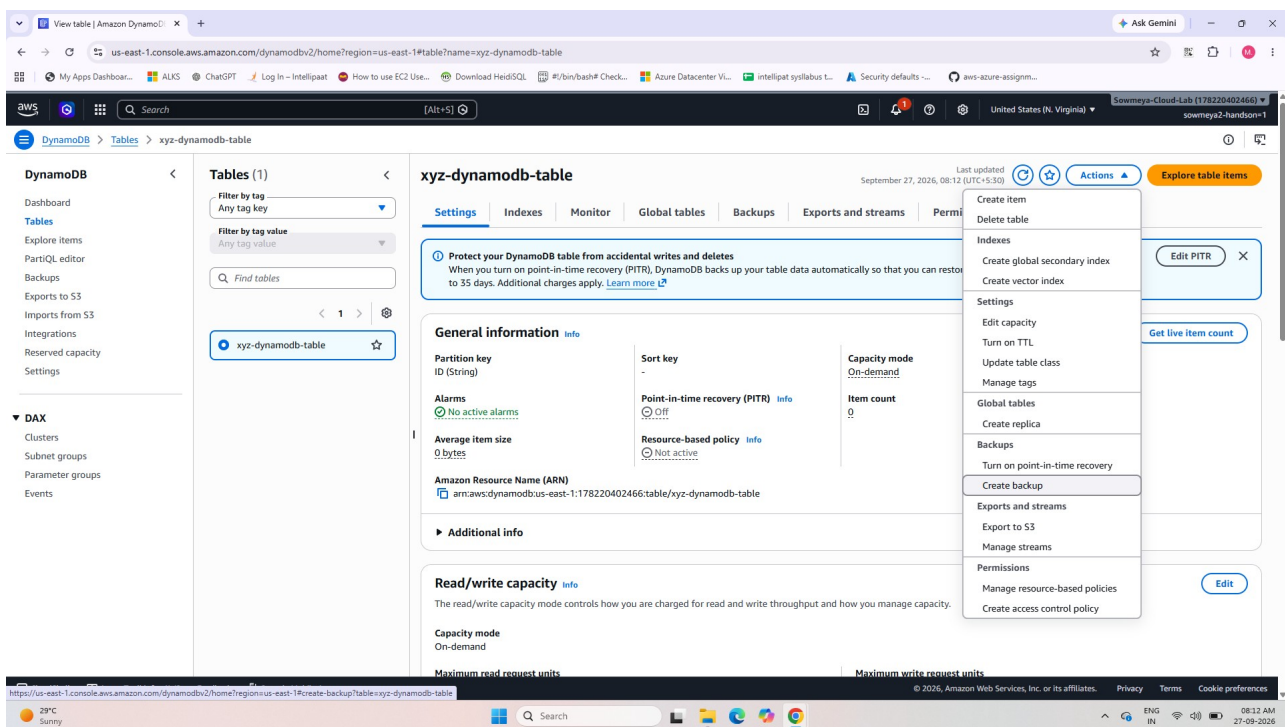


Figure 15: Actions → Create backup on xyz-dynamodb-table

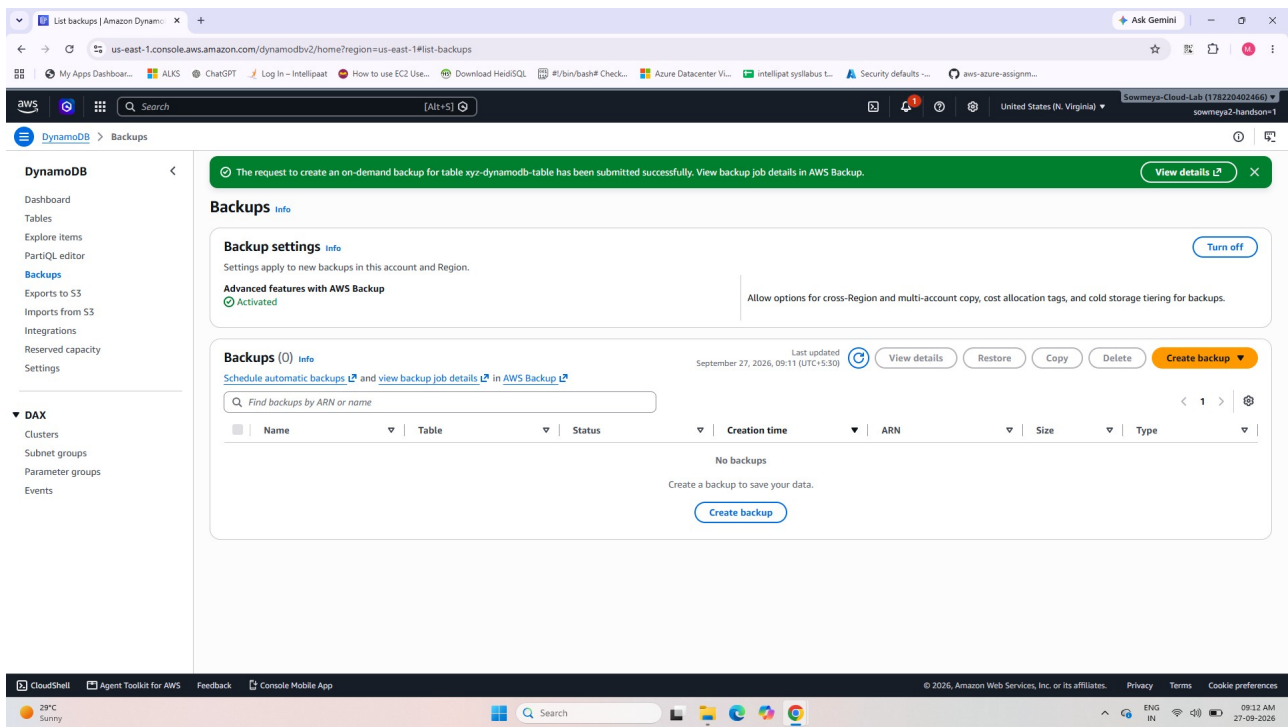


Figure 16: On-demand backup request submitted (AWS Backup advanced features activated)

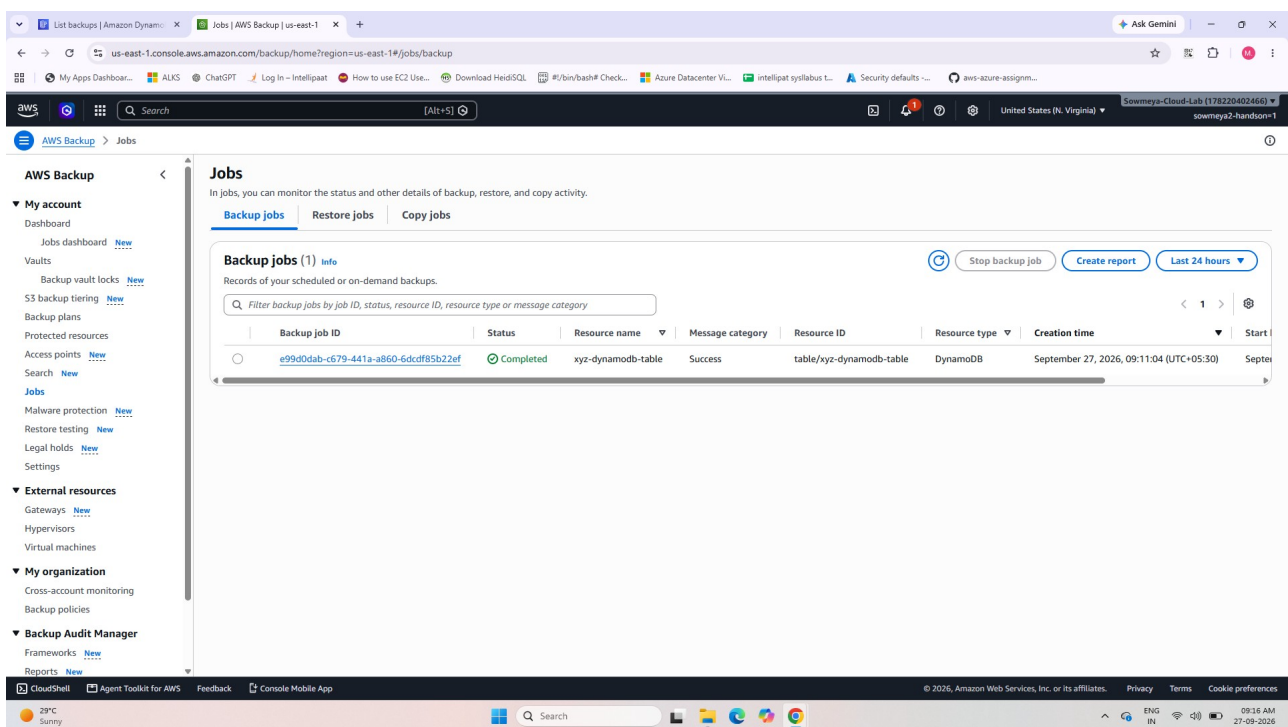


Figure 17: AWS Backup job for xyz-dynamodb-table – Completed

Why the DynamoDB Backups list shows 0: With AWS Backup advanced features turned on, on-demand backups are stored as recovery points in the AWS Backup vault, so they appear in AWS Backup rather than in the DynamoDB Backups list.

Step 4: Delete the table

1. In **DynamoDB → Tables**, select **xyz-dynamodb-table** and click **Delete**.
2. Leave **Delete all CloudWatch alarms** ticked. The backup already exists, so the extra pre-deletion backup is not needed.

3. Type **confirm** and click **Delete**.

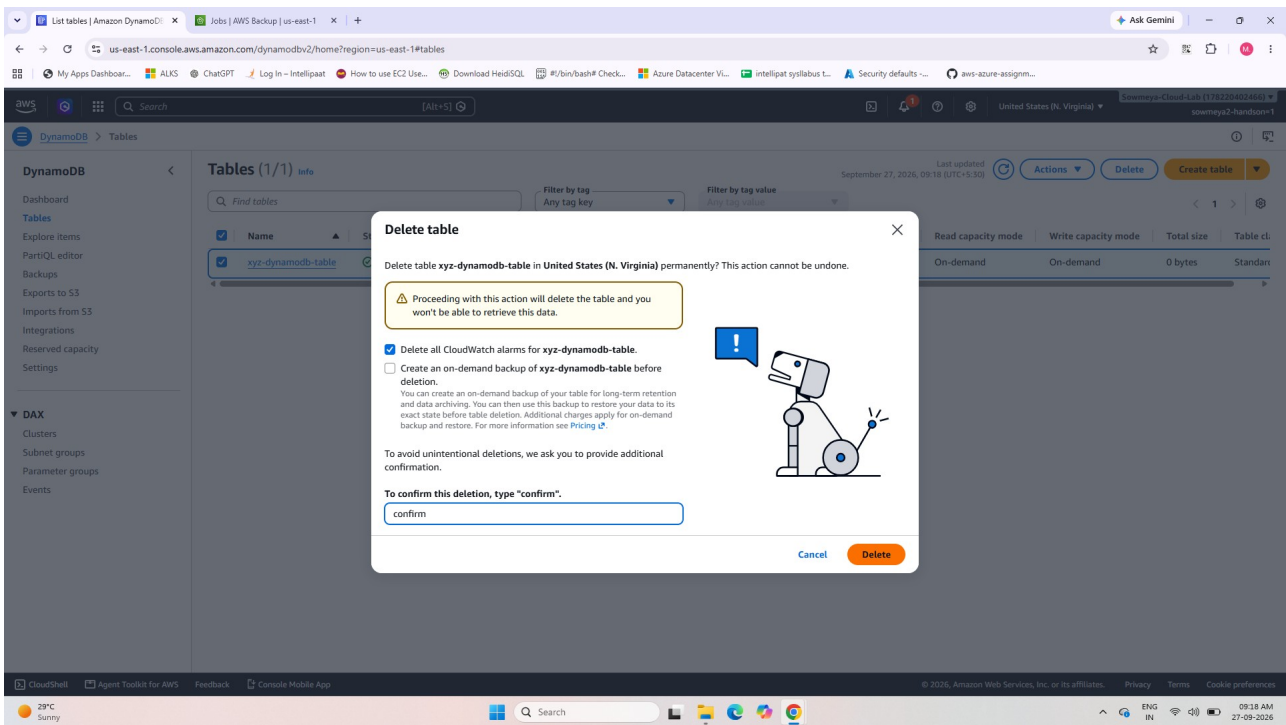


Figure 18: Delete table dialog for xyz-dynamodb-table

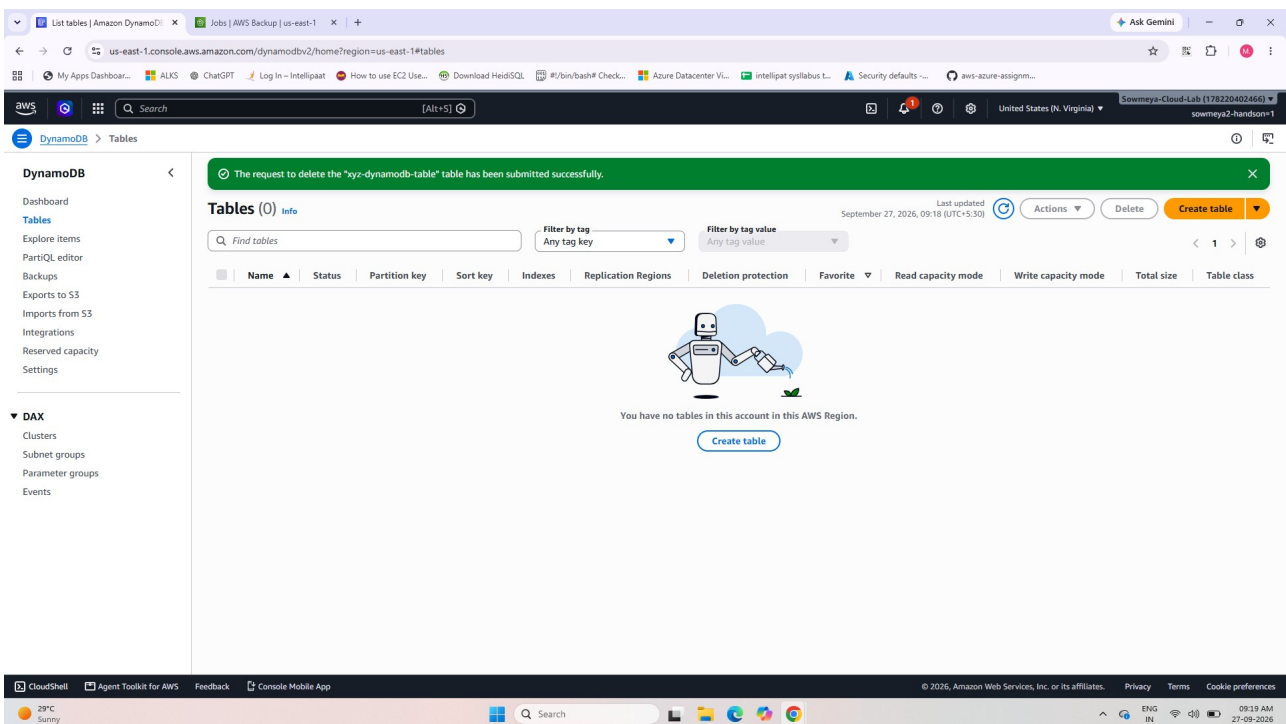


Figure 19: Table deleted – no tables left in us-east-1

Step 5: Confirm the backup is still available

Open **AWS Backup** → **Vaults** → **Default**. The recovery point for xyz-dynamodb-table is still there after the table was deleted, so the table can be restored at any time with **Actions** → **Restore**.

Default info

Summary

Vault name: Default

KMS encryption key ID: 51496d84-33ea-4bc7-b57c-e2826634ec50

Vault type: Backup vault

Creation date: September 27, 2026, 08:13:09 (UTC+05:30)

Vault lock: -

Retention period for vault lock: Minimum retention period: - Maximum retention period: -

Vault ARN: arn:aws:backup:us-east-1:178220402466:backup-vault:Default

Recovery points (1) info

Recovery point count value displayed in the console can be an approximation. See ListRecoveryPointsByBackupVault API to obtain the exact count.

Filter by resource type, recovery point ARN, status, resource ARN or source account ID

Recovery point ID	Status	Resource name	Resource ID	Resource type	Backup type	Creation time
05d50cfd-8465-47f1-9ee7-ee7e3fd626b	Completed	xyz-dynamodb-table	table/xyz-dynamodb-table	DynamoDB	Backup	September 27, 2026, 09:11:04 (UTC+05:30)

Access policy

Configure access to your Backup vault and protected resources by managing access policy permissions.

No permissions

No permissions to display.

Figure 20: Default backup vault – recovery point for xyz-dynamodb-table (Completed)

Jobs

In jobs, you can monitor the status and other details of backup, restore, and copy activity.

Backup jobs (1) info

Records of your scheduled or on-demand backups.

Filter backup jobs by job ID, status, resource ID, resource type or message category

Backup job ID	Status	Resource name	Message category	Resource ID	Resource type	Creation time	Start time
e99d0dab-c679-441a-a860-6dcdf85b22ef	Completed	xyz-dynamodb-table	Success	table/xyz-dynamodb-table	DynamoDB	September 27, 2026, 09:11:04 (UTC+05:30)	September 27, 2026, 09:11:04 (UTC+05:30)

Figure 21: AWS Backup jobs – backup job still listed as Completed after deletion

Assignment 4: Amazon Redshift

Problem statement: XYZ Corporation's application requires a database service that can store data which can be retrieved when required. Implement a suitable service.

Tasks:

1. Create a Redshift data warehouse.
2. Using the query editor: (a) load some data, (b) query the data.

Step 1: Create the Redshift cluster

1. Open **Amazon Redshift** → **Provisioned clusters dashboard** → **Create cluster**.
2. Cluster identifier: **xyz-redshift-cluster**. Node type: **ra3.large**. Number of nodes: **1**.
3. Database configurations: set the admin user name and password (database name **dev**).
4. Keep the default network settings and click **Create cluster**. Wait until the status is **Available**.

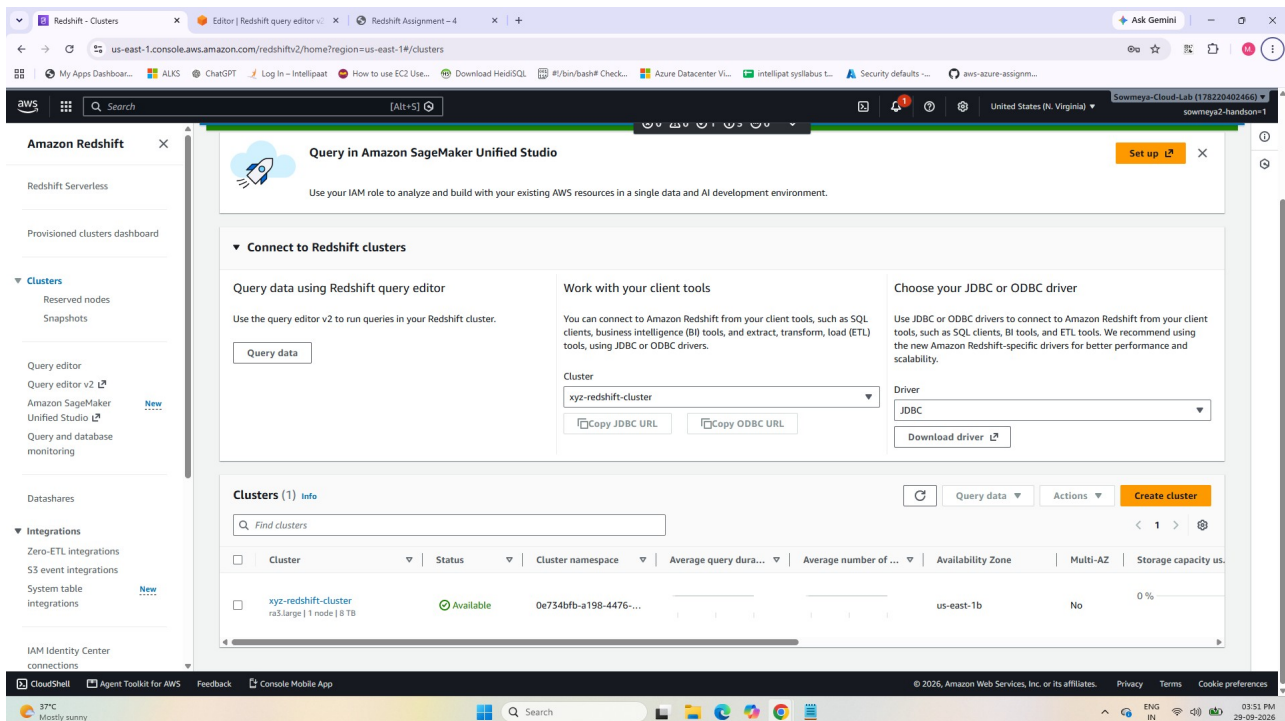


Figure 22: xyz-redshift-cluster Available (ra3.large, 1 node, us-east-1b)

Step 2: Open Query Editor v2 and connect

1. Click **Query data** → **Query in query editor v2**.
2. In the tree, click **xyz-redshift-cluster**, choose **Database user name and password**, enter database **dev** and the admin credentials, then **Create connection**.

Step 3: Load data

Create a table and insert five rows by running this script against the dev database:

```
CREATE TABLE employees (
  id          INT,
  name        VARCHAR(50),
  department  VARCHAR(50),
  salary      INT
);

INSERT INTO employees VALUES
(1, 'Ravi', 'IT', 60000),
(2, 'Priya', 'HR', 50000),
(3, 'Arun', 'IT', 75000),
(4, 'Meena', 'Finance', 65000),
(5, 'Kumar', 'Sales', 55000);
```

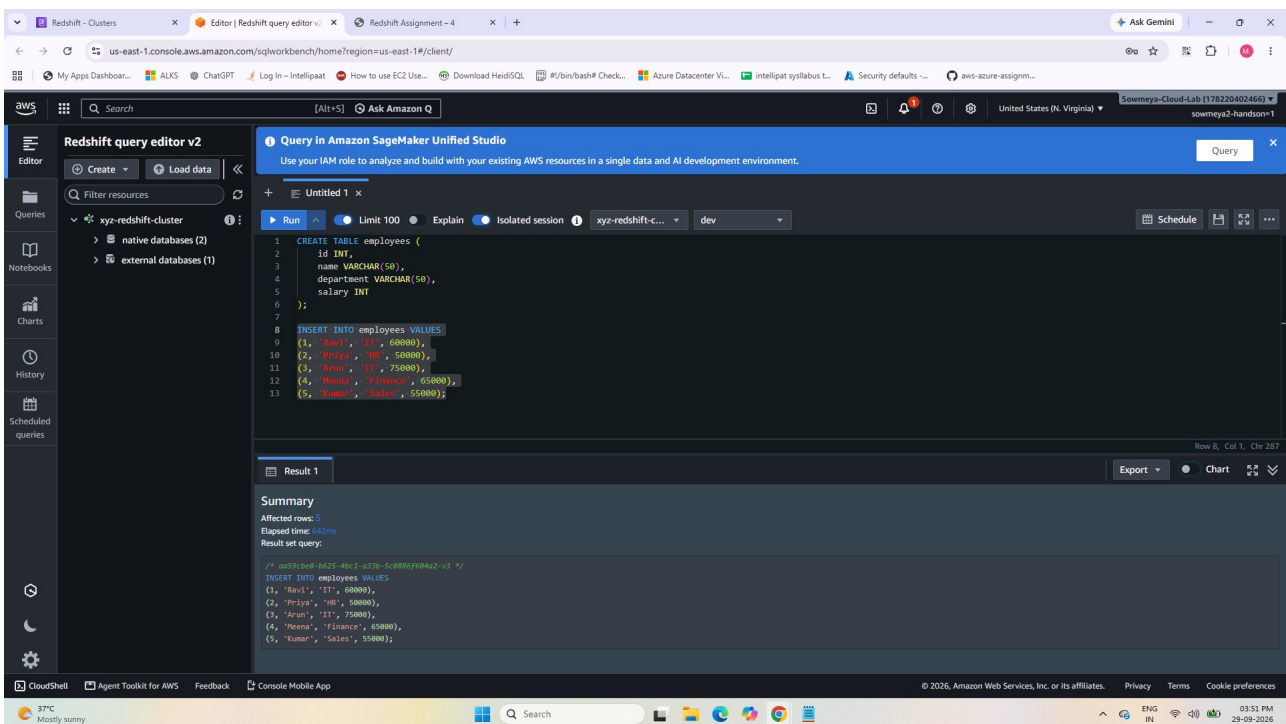


Figure 23: Query Editor v2 – employees table created and 5 rows inserted (Affected rows: 5)

Step 4: Query the data

Run queries against the loaded table, for example:

```
SELECT * FROM employees ORDER BY id;

SELECT department, COUNT(*) AS employees, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;
```

The first query returns all five rows; the second groups employees by department and shows the average salary for each.

Conclusion

Each task maps to the configuration and evidence below:

Assignment / task	How it was met	Evidence
1.1 MariaDB RDS database	xyz-mariadb-db, MariaDB 11.8, db.t3.micro, us-east-1a	Figures 1, 2
1.2a Windows SQL client	MySQL Workbench connection XYZ-MariaDB-RDS to the RDS endpoint	Figure 6
1.2b Linux EC2 client	XYZ-Linux-DB-Client connected with the mariadb client over SSL	Figures 4, 7
2.1 Aurora database	xyz-aurora-cluster (Aurora MySQL), writer in us-east-1d	Figure 8
2.2 Two read replicas	Readers in us-east-1c and us-east-1b	Figures 9–11
3.1 DynamoDB table	xyz-dynamodb-table with partition key ID (String)	Figures 12, 13
3.2 Five items	IDs 1–5 with Name and Department	Figure 14
3.3 Backup and delete	AWS Backup recovery point created, then table deleted	Figures 15–21
4.1 Redshift data warehouse	xyz-redshift-cluster, ra3.large, 1 node	Figure 22
4.2a Load data	employees table created and 5 rows inserted in Query Editor v2	Figure 23
4.2b Query data	SELECT queries on the employees table	Step 4 (Assignment 4)